МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ОТЧЁТ

по лабораторной работе № 5

«Хеширование и экспериментальное исследование методов разрешения коллизий»

Дисциплина: **Теория алгоритмов**

Направление подготовки: 38.03.05 «Бизнес-информатика»

Вариант 13

Выполнила: студентка группы БИ 2-1

Кугушева Арина Вячеславовна

Проверил: _____

2026

1. Цель работы и задание варианта

Цель работы: изучить хеширование как способ организации данных со временем доступа O(1) в среднем, реализовать хеш-таблицу и экспериментально исследовать влияние хеш-функции и коэффициента заполнения на её производительность.

Задание (вариант 13). Реализовать хеш-таблицу для целых ключей, **кратных 100**, с хеш-функцией метода деления h(k) = k mod m (m — простое) и двумя способами разрешения коллизий: **методом цепочек** и **двойным хешированием**. Вставить n = 30 000 элементов в таблицу размера m = 49 999, измерить время вставки и поиска, среднее число проб, распределение длин цепочек; сопоставить собственную реализацию со встроенным `dict` и с теоретическими кривыми. Дополнительное задание **B6**: исследовать кластеризацию при линейном пробировании — построить распределение длин непрерывных занятых участков при разных коэффициентах заполнения.

№	Тип ключей	Хеш	Коллизии	n	m	α = n/m	Доп.
13	целые кратные 100	Д (деление)	Ц и Дв	30 000	49 999	0,600	B6

Условия эксперимента. Ключи — различные числа вида 100·j, j — случайное целое из [1; 10^7) (генератор с фиксированным начальным значением 13). Выполняется по 10 000 успешных и 10 000 неуспешных запросов (во втором случае ключи кратны 100, но в таблице отсутствуют). Время вставки — медиана по 5 повторам построения таблицы, время запроса — медиана по 5 повторам серии. «Проба» — просмотр одной ячейки при открытой адресации и сравнение одного ключа при просмотре цепочки.

Кроме задания варианта выполнены общие пункты порядка работы: график числа проб в зависимости от α с теоретическими кривыми (α = 0,1…0,95), сравнение с хорошей и заведомо неудачной (m = 50 000, кратным 100) таблицей, проверка удаления с пометкой «удалено».

2. Хеш-функция и способы разрешения коллизий

Метод деления. h(k) = k mod m. Размер m = 49 999 — простое число (проверено: нет делителей до √49 999 ≈ 223,6). Это существенно именно для ключей, кратных 100: если бы m имело общий делитель d с числом 100, ключи 100·j занимали бы лишь m/d ячеек (при m = 50 000 — всего 500; см. раздел 6.2). Простое m, не делящее 100, обеспечивает использование всех ячеек.

Метод цепочек. Ячейка i хранит список пар (ключ, значение) с h(k) = i. Коэффициент заполнения α = n/m — средняя длина цепочки. Среднее число сравнений: при неуспешном поиске ≈ α, при успешном ≈ 1 + α/2; трудоёмкость Θ(1 + α), в худшем случае Θ(n). Допускается α > 1.

Двойное хеширование. Все элементы лежат в массиве; i-я проба для ключа k:

$$pos_i = (h_1(k) + i \cdot h_2(k)) \bmod m,\ h_1(k) = k \bmod m,\ h_2(k) = 1 + k \bmod (m - 2).$$

Шаг h_2 лежит в диапазоне [1; m − 2], зависит от ключа и, поскольку m простое, взаимно прост с m — последовательность проб обходит все ячейки. (Делитель m − 2 = 49 997 взят взаимно простым с 100, чтобы шаги ключей, кратных 100, не вырождались.) Теоретическое среднее число проб при равномерном хешировании: неуспешный поиск 1/(1 − α), успешный (1/α)·ln(1/(1 − α)). Обязательно α < 1.

Линейное пробирование (для задания B6): шаг равен 1. Оно приводит к первичной кластеризации — слиянию занятых участков в длинные кластеры. Оценки Кнута:

неуспешный поиск $\frac{1}{2}(1 + 1/(1 - \alpha)^2)$, успешный $\frac{1}{2}(1 + 1/(1 - \alpha))$.

Удаление. В цепочке элемент просто удаляется из списка. При открытой адресации ячейка помечается специальной меткой «удалено» (tombstone): простое обнуление разорвало бы последовательности проб других ключей, проходящих через эту ячейку.

Таблица 1. Теоретические оценки среднего числа проб при $\alpha = 0{,}60$

Способ	Неуспешный поиск	Успешный поиск
Метод цепочек	0,600	1,300
Двойное хеширование	2,500	1,527
Линейное пробирование	3,625	1,750

3. Схемы структуры данных и блок-схемы алгоритмов

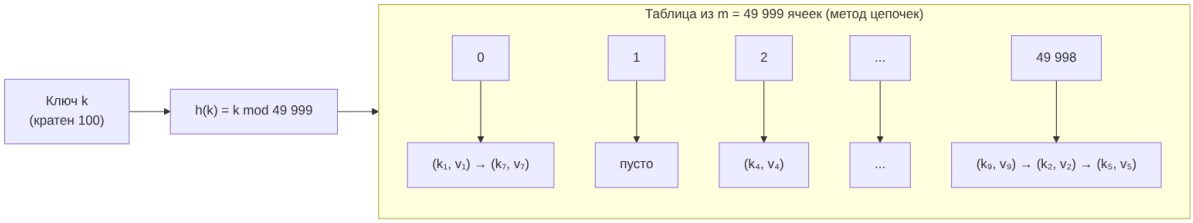


Рис. 1. Схема хеш-таблицы с методом цепочек

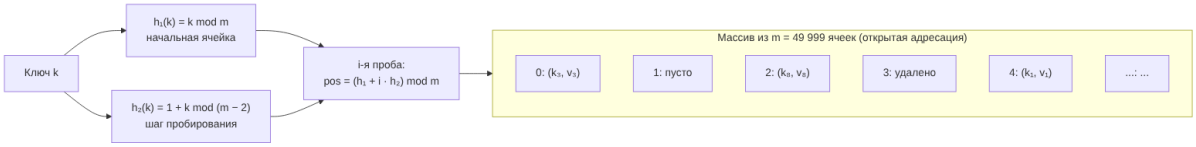


Рис. 2. Схема хеш-таблицы с открытой адресацией (двойное хеширование)

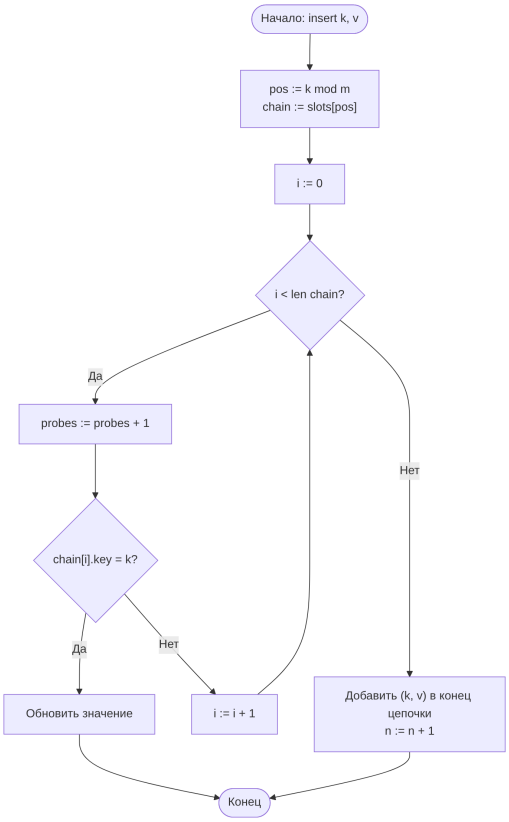


Рис. 3. Блок-схема вставки при методе цепочек

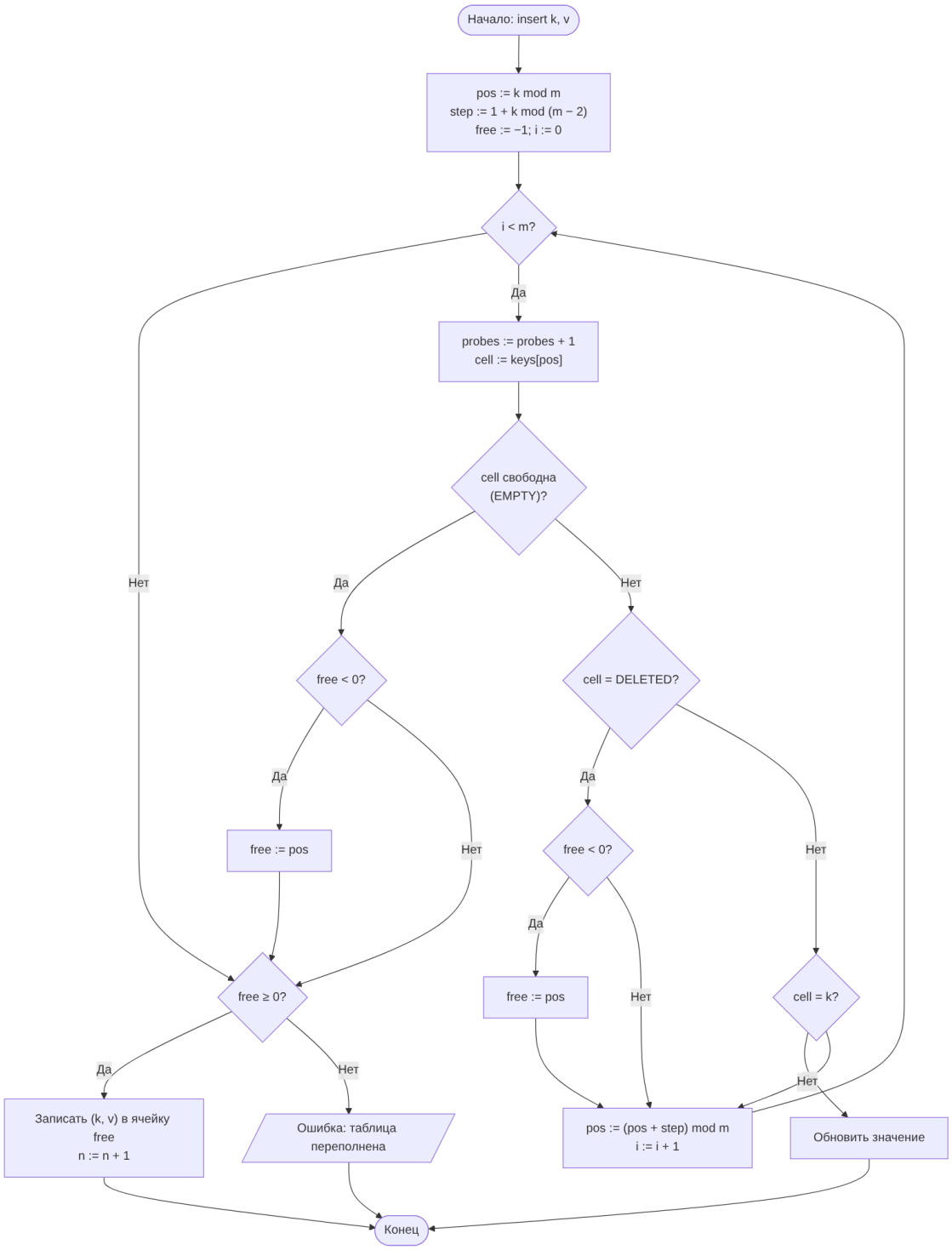


Рис. 4. Блок-схема вставки при двойном хешировании (с учётом меток «удалено»)

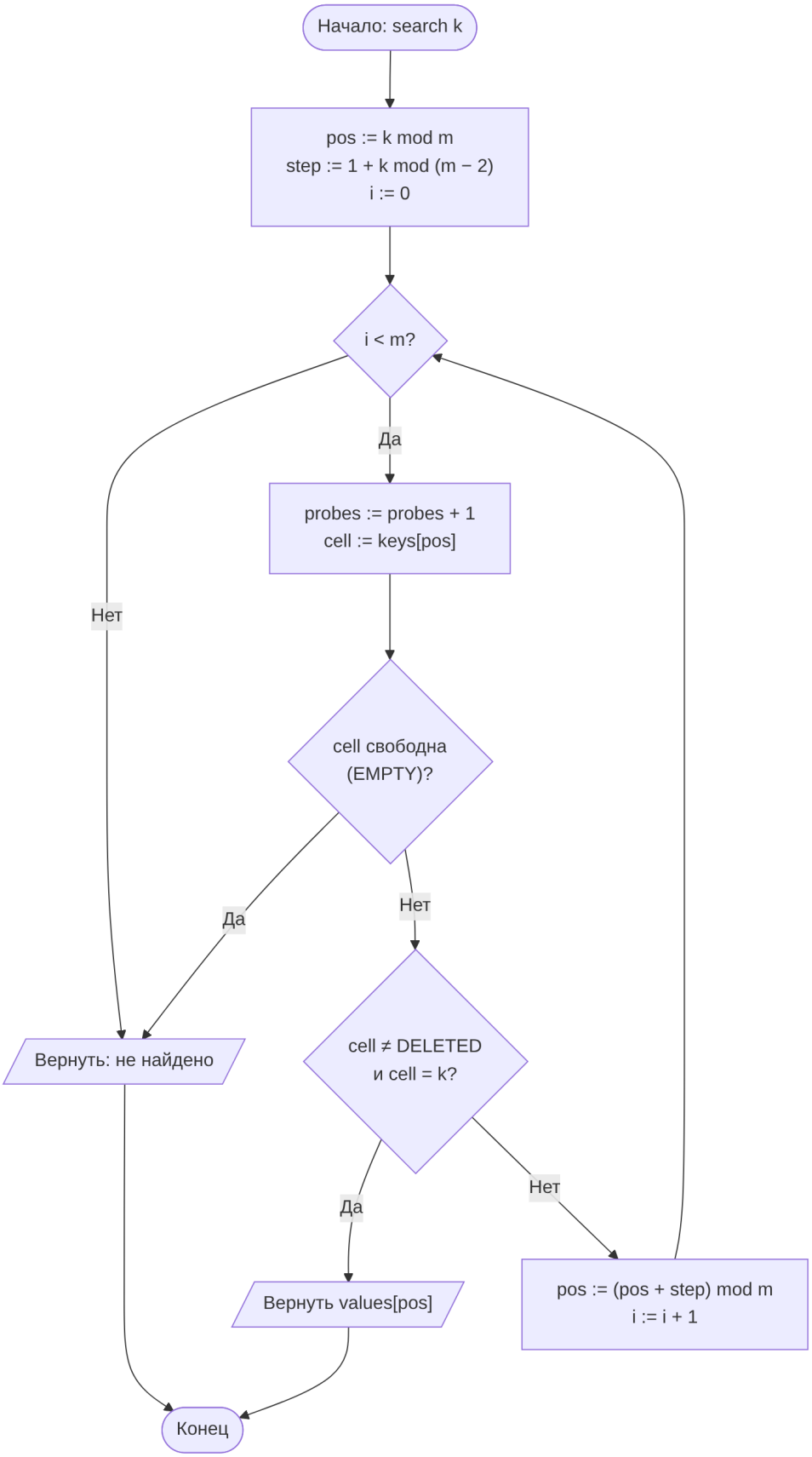


Рис. 5. Блок-схема поиска при двойном хешировании

4. Листинг программы

Файл `lr5_variant13.py` (самодостаточный модуль с подробными комментариями: таблицы, генерация данных, измерения, самопроверка, эксперименты, графики). Запуск: `python lr5_variant13.py`; требуется matplotlib.

```
"""
Лабораторная работа № 5. Вариант 13.
Дисциплина: Теория алгоритмов.
Выполнила: Кугушева Арина Вячеславовна, группа БИ 2-1.

Тема: хеширование и экспериментальное исследование методов разрешения коллизий.

ИСХОДНЫЕ ДАННЫЕ ВАРИАНТА 13 (табл. 1 задания)
    тип ключей ............. целые числа, кратные 100;
    хеш-функция ............ метод деления h(k) = k mod m, где m - простое;
    разрешение коллизий .... метод цепочек и двойное хеширование;
    число элементов n ...... 30 000;
    размер таблицы m ....... 49 999 (простое число);
    коэффициент заполнения . alpha = n / m = 0,6000...;
    дополнительное задание . В6 - исследование КЛАСТЕРИЗАЦИИ при линейном
                              пробировании: распределение длин непрерывных
                              занятых участков при разных alpha.

ЧТО ДЕЛАЕТ ПРОГРАММА (по порядку)
    0. Самопроверка: все три таблицы сверяются со встроенным dict
       (вставка, поиск, удаление, обновление значения).
    1. Основной эксперимент (n = 30 000, m = 49 999): время вставки и поиска,
       среднее число проб при успешном и неуспешном поиске для
       - метода цепочек,
       - двойного хеширования,
       - линейного пробирования (нужно для В6 и как точка сравнения),
       - встроенного dict (эталон).
    2. Распределение длин цепочек (сравнение с законом Пуассона) и
       демонстрация того, почему размер m должен быть простым: для ключей,
       кратных 100, таблица с m = 50 000 использует лишь 500 ячеек.
    3. Зависимость числа проб от alpha (0,1 ... 0,95) для линейного
       пробирования и двойного хеширования + теоретические кривые.
    4. ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ В6: распределение длин кластеров при
       линейном пробировании для alpha = 0,3; 0,5; 0,6; 0,7; 0,9 и сравнение
       с двойным хешированием.
    5. Удаление: показывает, что обнуление ячейки при открытой адресации
       «теряет» ключи, а пометка «удалено» (tombstone) - нет.

ЗАПУСК
    python lr5_variant13.py             (полный прогон, около 1-2 минут)
    python lr5_variant13.py --json F    (дополнительно сохранить результаты в F)
    python lr5_variant13.py --show      (дополнительно показать графики)

Требуется: Python 3.10+, библиотека matplotlib (pip install matplotlib).
Результат: таблицы в консоли и графики lr5_plot_probes.png,
lr5_plot_chains.png, lr5_plot_clusters.png в текущей папке.
"""

import json
import math
import platform
import random
import statistics
import sys
import time
from collections import Counter

import matplotlib

if "--show" not in sys.argv:
    matplotlib.use("Agg")          # режим без окна: графики только сохраняются в файлы
import matplotlib.pyplot as plt

# ---------------------------------------------------------------------------
# 1. ПАРАМЕТРЫ ЭКСПЕРИМЕНТА
# ---------------------------------------------------------------------------
N = 30_000                  # число хранимых элементов (вариант 13)
M = 49_999                  # размер таблицы - простое число (вариант 13)
M_BAD = 50_000              # «плохой» размер: делится на 100 (для демонстрации)
STEP_KEYS = 100             # ключи кратны 100 (вариант 13)
KEY_LIMIT = 10_000_000      # ключи выбираются как 100 * j, где j из [1; KEY_LIMIT)
SEED = 13                   # начальное значение генератора случайных чисел
N_QUERIES = 10_000          # число запросов каждого вида (успешных / неуспешных)
REPS = 5                    # число повторов при измерении времени (берём медиану)
ALPHAS_SWEEP = [0.1, 0.3, 0.5, 0.7, 0.9, 0.95]   # коэффициенты заполнения для п. 3
SWEEP_SEEDS = 3             # число независимых наборов ключей для усреднения в п. 3
```

```
SWEEP_QUERIES = 5_000       # число запросов каждого вида в п. 3
ALPHAS_CLUSTER = [0.3, 0.5, 0.6, 0.7, 0.9]       # коэффициенты заполнения для В6
CLUSTER_BINS = [(1, 1), (2, 2), (3, 4), (5, 8), (9, 16), (17, 32), (33, 64), (65, None)]
                            # интервалы длин кластеров для таблицы В6


# ---------------------------------------------------------------------------
# 2. ХЕШ-ТАБЛИЦЫ
# ---------------------------------------------------------------------------
class ChainHashTable:
    """
    Хеш-таблица с разрешением коллизий МЕТОДОМ ЦЕПОЧЕК.

    Структура: массив из m ячеек; в ячейке с номером i лежит список (цепочка)
    пар (ключ, значение), у которых h(k) = i.
    Хеш-функция - метод деления: h(k) = k mod m.

    Счётчик self.probes считает число СРАВНЕНИЙ ключей, выполненных при
    просмотре цепочек («проб»). Обращение к ячейке массива за пробу не
    считается, поэтому при неуспешном поиске в среднем получается alpha
    проб, при успешном - примерно 1 + alpha / 2.
    """

    def __init__(self, m=M):
        self.m = m                                  # число ячеек таблицы
        self.slots = [[] for _ in range(m)]         # m пустых цепочек
        self.n = 0                                  # число хранимых элементов
        self.probes = 0                             # счётчик проб (сравнений ключей)

    def _hash(self, key):
        """Хеш-функция метода деления: остаток от деления ключа на m."""
        return key % self.m

    def insert(self, key, value):
        """
        Вставка пары (key, value). Если ключ уже есть - значение обновляется.
        Возвращает True, если добавлен новый ключ, и False при обновлении.
        """
        chain = self.slots[self._hash(key)]         # цепочка нужной ячейки
        for i, (k, _) in enumerate(chain):          # ищем ключ в цепочке
            self.probes += 1
            if k == key:                            # ключ уже есть - обновляем
                chain[i] = (key, value)
                return False
        chain.append((key, value))                  # ключа нет - добавляем в конец
        self.n += 1
        return True

    def search(self, key):
        """Поиск по ключу. Возвращает значение или None, если ключа нет."""
        for k, v in self.slots[self._hash(key)]:    # просматриваем всю цепочку
            self.probes += 1
            if k == key:
                return v
        return None

    def delete(self, key):
        """Удаление ключа. Возвращает True, если ключ был найден и удалён."""
        chain = self.slots[self._hash(key)]
        for i, (k, _) in enumerate(chain):
            self.probes += 1
            if k == key:
                del chain[i]                        # в цепочке удаление безопасно
                self.n -= 1
                return True
        return False

    @property
    def load_factor(self):
        """Коэффициент заполнения alpha = n / m (средняя длина цепочки)."""
        return self.n / self.m

    def chain_lengths(self):
        """Список длин всех m цепочек (для гистограммы)."""
        return [len(chain) for chain in self.slots]


class OpenAddressTable:
    """
    Хеш-таблица с ОТКРЫТОЙ АДРЕСАЦИЕЙ (все элементы лежат прямо в массиве).

    Последовательность проб для ключа k:
        pos_i = (h1(k) + i * step(k)) mod m,    i = 0, 1, 2, ...
    где h1(k) = k mod m - основная хеш-функция (метод деления), а шаг
        - линейное пробирование (mode = "linear"): step(k) = 1;
```

```
    - двойное хеширование (mode = "double"):
          step(k) = h2(k) = 1 + (k mod (m - 2)),
      то есть шаг берётся из диапазона [1; m - 2] и зависит от ключа.
      Так как m простое, любой шаг от 1 до m - 1 взаимно прост с m, и
      последовательность проб проходит по ВСЕМ ячейкам таблицы.

Состояния ячейки: EMPTY (никогда не занималась), DELETED (была занята,
ключ удалён - «надгробие», tombstone) либо хранится ключ.

Счётчик self.probes считает число просмотренных ячеек (проб).
"""

EMPTY = object()        # метка «ячейка свободна»
DELETED = object()      # метка «ячейка освобождена удалением»

def __init__(self, m=M, mode="double"):
    if mode not in ("linear", "double"):
        raise ValueError("mode должен быть 'linear' или 'double'")
    self.m = m
    self.mode = mode
    self.keys = [self.EMPTY] * m                # ключи (или метки EMPTY / DELETED)
    self.values = [None] * m                    # значения, лежащие рядом с ключами
    self.n = 0                                  # число живых элементов
    self.probes = 0                             # счётчик проб (просмотренных ячеек)

def _start_and_step(self, key):
    """Возвращает (начальная позиция, шаг пробирования) для ключа."""
    start = key % self.m                        # h1(k) - метод деления
    if self.mode == "linear":
        return start, 1                         # линейное пробирование: шаг 1
    return start, 1 + key % (self.m - 2)        # двойное хеширование: h2(k)

def insert(self, key, value):
    """
    Вставка пары (key, value); при совпадении ключа значение обновляется.
    Возвращает True для нового ключа, False при обновлении.

    Сначала проходим всю последовательность проб до первой ЧИСТО свободной
    ячейки (EMPTY): ключ мог лежать за «надгробием», и дубликат создавать
    нельзя. Новый ключ записывается в первую встреченную свободную или
    помеченную DELETED ячейку - так «надгробия» используются повторно.
    """
    pos, step = self._start_and_step(key)
    first_free = -1                             # первая подходящая для записи ячейка
    for _ in range(self.m):                     # не более m проб
        self.probes += 1
        cell = self.keys[pos]
        if cell is self.EMPTY:                  # дальше ключа быть не может
            if first_free < 0:
                first_free = pos
            break
        if cell is self.DELETED:                # запоминаем, но идём дальше
            if first_free < 0:
                first_free = pos
        elif cell == key:                       # ключ найден - обновляем значение
            self.values[pos] = value
            return False
        pos = (pos + step) % self.m             # следующая проба
    if first_free < 0:                          # ни одной свободной ячейки
        raise RuntimeError("Таблица переполнена")
    self.keys[first_free] = key
    self.values[first_free] = value
    self.n += 1
    return True

def _find(self, key):
    """Возвращает индекс ячейки с ключом или -1, если ключа нет."""
    pos, step = self._start_and_step(key)
    for _ in range(self.m):
        self.probes += 1
        cell = self.keys[pos]
        if cell is self.EMPTY:                  # дошли до свободной ячейки - ключа нет
            return -1
        if cell is not self.DELETED and cell == key:
            return pos
        pos = (pos + step) % self.m             # «надгробия» пропускаем
    return -1                                   # обошли все ячейки

def search(self, key):
    """Поиск по ключу. Возвращает значение или None."""
    pos = self._find(key)
    return None if pos < 0 else self.values[pos]

def delete(self, key, tombstone=True):
    """
```

```
        Удаление ключа. Возвращает True, если ключ был найден.

        tombstone=True  - правильный способ: ячейка помечается DELETED, цепочка
                          проб других ключей не разрывается.
        tombstone=False - НЕПРАВИЛЬНЫЙ способ (простое обнуление ячейки), он
                          нужен только для демонстрации ошибки: поиск ключей,
                          лежащих дальше по последовательности проб, обрывается.
        """
        pos = self._find(key)
        if pos < 0:
            return False
        self.keys[pos] = self.DELETED if tombstone else self.EMPTY
        self.values[pos] = None
        self.n -= 1
        return True

    @property
    def load_factor(self):
        """Коэффициент заполнения alpha = n / m (по живым элементам)."""
        return self.n / self.m

    def cluster_lengths(self):
        """
        Длины кластеров - непрерывных участков занятых ячеек массива
        (ячейки EMPTY разделяют кластеры; ячейка DELETED считается занятой).
        Переход через конец массива к началу не учитывается.
        """
        lengths, run = [], 0                        # run - длина текущего участка
        for cell in self.keys:
            if cell is not self.EMPTY:              # ячейка занята
                run += 1
            elif run:                               # участок закончился
                lengths.append(run)
                run = 0
        if run:                                     # участок дошёл до конца массива
            lengths.append(run)
        return lengths


# ---------------------------------------------------------------------------
# 3. ГЕНЕРАЦИЯ ДАННЫХ
# ---------------------------------------------------------------------------
def make_keys(n, rng):
    """n РАЗЛИЧНЫХ случайных ключей, кратных 100 (вариант 13)."""
    return [STEP_KEYS * j for j in rng.sample(range(1, KEY_LIMIT), n)]


def make_missing(present, count, rng):
    """
    count ключей, кратных 100, которых НЕТ в множестве present
    (для неуспешного поиска). Подходящий ключ получается отбраковкой.
    """
    missing = []
    while len(missing) < count:
        k = STEP_KEYS * rng.randrange(1, KEY_LIMIT)
        if k not in present:                        # ключ не должен входить в таблицу
            missing.append(k)
    return missing


# ---------------------------------------------------------------------------
# 4. ИЗМЕРЕНИЯ
# ---------------------------------------------------------------------------
def median_time(func, reps=REPS):
    """Время работы func() в секундах: медиана по reps повторам."""
    times = []
    for _ in range(reps):
        t0 = time.perf_counter()
        func()
        times.append(time.perf_counter() - t0)
    return statistics.median(times)


def avg_probes(table, queries):
    """Среднее число проб на один запрос из списка queries."""
    table.probes = 0                                # обнуляем счётчик
    for q in queries:
        table.search(q)
    return table.probes / len(queries)


def benchmark(factory, keys, found_q, missing_q):
    """
    Полное измерение одной реализации.
    factory   - функция без аргументов, создающая пустую таблицу;
```

```
    keys      - ключи для вставки;
    found_q   - запросы, ключи которых есть в таблице (успешный поиск);
    missing_q - запросы, ключей которых нет (неуспешный поиск).

    Возвращает словарь: время вставки всех ключей, время одного успешного и
    одного неуспешного запроса (секунды), среднее число проб в обоих случаях.
    """
    def build():
        table = factory()
        for k in keys:
            table.insert(k, k)
        return table

    t_insert = median_time(build)                   # время вставки n ключей
    table = build()                                 # таблица для измерения поиска
    t_found = median_time(lambda: [table.search(q) for q in found_q]) / len(found_q)
    t_missing = median_time(lambda: [table.search(q) for q in missing_q]) / len(missing_q)
    return {
        "t_insert": t_insert,
        "t_found": t_found,
        "t_missing": t_missing,
        "probes_found": avg_probes(table, found_q),
        "probes_missing": avg_probes(table, missing_q),
    }


def benchmark_dict(keys, found_q, missing_q):
    """То же измерение для встроенного словаря dict (эталон)."""
    def build():
        d = {}
        for k in keys:                              # вставка тем же циклом, что и у таблиц
            d[k] = k
        return d

    t_insert = median_time(build)
    d = build()
    t_found = median_time(lambda: [d.get(q) for q in found_q]) / len(found_q)
    t_missing = median_time(lambda: [d.get(q) for q in missing_q]) / len(missing_q)
    return {"t_insert": t_insert, "t_found": t_found, "t_missing": t_missing}


# ---------------------------------------------------------------------------
# 5. ТЕОРЕТИЧЕСКИЕ ОЦЕНКИ ЧИСЛА ПРОБ
# ---------------------------------------------------------------------------
def theory_chain(alpha):
    """Метод цепочек: (успешный, неуспешный) - 1 + alpha/2 и alpha сравнений ключей."""
    return 1 + alpha / 2, alpha


def theory_double(alpha):
    """Двойное хеширование (равномерное): успешный (1/a)ln(1/(1-a)), неуспешный 1/(1-a)."""
    return math.log(1 / (1 - alpha)) / alpha, 1 / (1 - alpha)


def theory_linear(alpha):
    """Линейное пробирование (Кнут): успешный (1+1/(1-a))/2, неуспешный (1+1/(1-a)^2)/2."""
    return 0.5 * (1 + 1 / (1 - alpha)), 0.5 * (1 + 1 / (1 - alpha) ** 2)


# ---------------------------------------------------------------------------
# 6. ЭКСПЕРИМЕНТЫ
# ---------------------------------------------------------------------------
def experiment_main():
    """Основной эксперимент варианта: n = 30 000, m = 49 999 (alpha = 0,6)."""
    rng = random.Random(SEED)
    keys = make_keys(N, rng)                        # 30 000 различных ключей, кратных 100
    present = set(keys)
    found_q = [rng.choice(keys) for _ in range(N_QUERIES)]      # успешные запросы
    missing_q = make_missing(present, N_QUERIES, rng)           # неуспешные запросы
    res = {
        "chain": benchmark(lambda: ChainHashTable(M), keys, found_q, missing_q),
        "double": benchmark(lambda: OpenAddressTable(M, "double"), keys, found_q, missing_q),
        "linear": benchmark(lambda: OpenAddressTable(M, "linear"), keys, found_q, missing_q),
        "dict": benchmark_dict(keys, found_q, missing_q),
    }
    alpha = N / M
    res["alpha"] = alpha
    res["theory"] = {"chain": theory_chain(alpha), "double": theory_double(alpha),
                     "linear": theory_linear(alpha)}
    return res


def experiment_chains():
    """
    Распределение длин цепочек при m = 49 999 (простое) и для сравнения при
```

```
    m = 50 000 (делится на 100): ключи, кратные 100, при таком m попадают
    только в 500 ячеек из 50 000.
    """
    rng = random.Random(SEED)
    keys = make_keys(N, rng)
    out = {}
    for name, m in (("prime", M), ("bad", M_BAD)):
        table = ChainHashTable(m)
        for k in keys:
            table.insert(k, k)
        lengths = table.chain_lengths()
        counts = Counter(lengths)                  # длина цепочки -> число ячеек
        table.probes = 0
        for q in keys[:N_QUERIES]:
            table.search(q)
        out[name] = {
            "m": m,
            "max": max(lengths),
            "empty": counts[0],
            "used": m - counts[0],
            "hist": {str(L): counts[L] for L in sorted(counts)},
            "probes_found": table.probes / N_QUERIES,
        }
    # теоретическое распределение Пуассона: m * e^(-a) * a^L / L!
    alpha = N / M
    out["poisson"] = {str(L): M * math.exp(-alpha) * alpha ** L / math.factorial(L)
                      for L in range(0, out["prime"]["max"] + 1)}
    return out


def experiment_sweep():
    """
    Среднее число проб при разных alpha для линейного пробирования и двойного
    хеширования (усреднение по SWEEP_SEEDS независимым наборам ключей).
    """
    out = {}
    for alpha in ALPHAS_SWEEP:
        n = round(alpha * M)                       # число ключей для данного alpha
        acc = {"linear": [[], []], "double": [[], []]}
        for s in range(SWEEP_SEEDS):
            rng = random.Random(1000 * s + int(alpha * 100))
            keys = make_keys(n, rng)
            present = set(keys)
            found_q = [rng.choice(keys) for _ in range(SWEEP_QUERIES)]
            missing_q = make_missing(present, SWEEP_QUERIES, rng)
            for mode in ("linear", "double"):
                table = OpenAddressTable(M, mode)
                for k in keys:
                    table.insert(k, k)
                acc[mode][0].append(avg_probes(table, found_q))
                acc[mode][1].append(avg_probes(table, missing_q))
        out[str(alpha)] = {
            "n": n,
            "linear": [statistics.mean(acc["linear"][0]), statistics.mean(acc["linear"][1])],
            "double": [statistics.mean(acc["double"][0]), statistics.mean(acc["double"][1])],
            "theory_linear": theory_linear(alpha),
            "theory_double": theory_double(alpha),
        }
    return out


def bin_label(lo, hi):
    """Подпись интервала длин кластеров: '1', '3-4', '65+'."""
    if hi is None:
        return f"{lo}+"
    return str(lo) if lo == hi else f"{lo}-{hi}"


def experiment_clusters():
    """
    ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ В6: кластеризация при линейном пробировании.

    Для каждого alpha строится таблица (n = alpha * m) и измеряются длины
    непрерывных занятых участков. Для сравнения то же делается для двойного
    хеширования. Если бы ячейки занимались независимо друг от друга с
    вероятностью alpha, средняя длина участка была бы 1 / (1 - alpha).
    """
    out = {}
    for alpha in ALPHAS_CLUSTER:
        n = N if abs(alpha - N / M) < 1e-3 else round(alpha * M)   # для alpha = 0,6 берём ровно N = 30 000
        rng = random.Random(SEED + int(alpha * 100))
        keys = make_keys(n, rng)                   # одни и те же ключи для обоих способов
        row = {"n": n}
        for mode in ("linear", "double"):
            table = OpenAddressTable(M, mode)
```

```
            for k in keys:
                table.insert(k, k)
            lengths = table.cluster_lengths()
            bins = []
            for lo, hi in CLUSTER_BINS:             # число кластеров в каждом интервале
                bins.append(sum(1 for L in lengths if L >= lo and (hi is None or L <= hi)))
            row[mode] = {
                "clusters": len(lengths),
                "mean": statistics.mean(lengths),
                "max": max(lengths),
                "bins": bins,
                "hist": dict(Counter(lengths)),     # полное распределение (для графика)
            }
        row["random_mean"] = 1 / (1 - alpha)        # средняя длина при независимом заполнении
        out[str(alpha)] = row
    return out


def experiment_delete():
    """
    Удаление при открытой адресации (линейное пробирование, alpha = 0,6).

    Удаляется 20 % ключей двумя способами: пометкой DELETED и простым
    обнулением ячейки. После этого ищем ВСЕ оставшиеся ключи и считаем, сколько
    из них «потерялось» (поиск вернул None, хотя ключ в таблице лежит).
    """
    rng = random.Random(SEED)
    keys = make_keys(N, rng)
    to_delete = set(rng.sample(keys, N // 5))
    remaining = [k for k in keys if k not in to_delete]
    out = {"deleted": len(to_delete), "remaining": len(remaining)}
    for mode in ("linear", "double"):
        for tomb in (True, False):
            table = OpenAddressTable(M, mode)
            for k in keys:
                table.insert(k, k)
            for k in to_delete:
                table.delete(k, tombstone=tomb)
            lost = sum(1 for k in remaining if table.search(k) is None)
            out[f"{mode}_{'tombstone' if tomb else 'zero'}"] = lost
    return out


# ---------------------------------------------------------------------------
# 7. САМОПРОВЕРКА
# ---------------------------------------------------------------------------
def self_test():
    """
    Сверяет три реализации со встроенным dict на случайных операциях
    вставки, обновления, удаления и поиска (небольшие таблицы, много коллизий).
    При ошибке выбрасывает AssertionError.
    """
    rng = random.Random(0)
    for m in (11, 101, 1009):                       # простые размеры
        tables = [ChainHashTable(m), OpenAddressTable(m, "linear"),
                  OpenAddressTable(m, "double")]
        ref = {}                                    # эталонный словарь
        limit = int(m * 0.7)                        # не заполняем таблицу целиком
        for _ in range(4000):
            op = rng.choice(["ins", "ins", "del", "get"])
            key = STEP_KEYS * rng.randrange(1, 3 * m)   # ключи тоже кратны 100
            if op == "ins" and (key in ref or len(ref) < limit):
                val = rng.random()
                ref[key] = val
                for t in tables:
                    t.insert(key, val)
            elif op == "del":
                expected = key in ref
                ref.pop(key, None)
                for t in tables:
                    assert t.delete(key) == expected, "delete вернул неверный результат"
            else:
                for t in tables:
                    assert t.search(key) == ref.get(key), "search вернул неверное значение"
        for t in tables:
            assert t.n == len(ref), "неверный счётчик элементов"
    print("Самопроверка пройдена (insert / search / delete совпадают с dict)")


# ---------------------------------------------------------------------------
# 8. ГРАФИКИ
# ---------------------------------------------------------------------------
def make_plots(res):
    """Строит и сохраняет три рисунка."""
    # --- рисунок 1: число проб в зависимости от alpha ----------------------
```

```
    sweep = res["sweep"]
    alphas = [float(a) for a in sweep]
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
    titles = ["Неуспешный поиск", "Успешный поиск"]
    for idx, ax in enumerate(axes):
        # теоретические кривые строим по плотной сетке alpha
        grid = [i / 200 for i in range(2, 192)]
        ax.plot(grid, [theory_linear(a)[idx] for a in grid], "r--", lw=1, label="линейное - теория")
        ax.plot(grid, [theory_double(a)[idx] for a in grid], "b--", lw=1, label="двойное - теория")
        ax.plot(alphas, [sweep[str(a)]["linear"][idx] for a in alphas], "ro-", label="линейное - опыт")
        ax.plot(alphas, [sweep[str(a)]["double"][idx] for a in alphas], "bs-", label="двойное - опыт")
        ax.set_yscale("log")
        ax.set_xlabel("Коэффициент заполнения α")
        ax.set_ylabel("Среднее число проб")
        ax.set_title(titles[idx])
        ax.grid(True, which="both", alpha=0.3)
        ax.legend(fontsize=8)
    fig.tight_layout()
    fig.savefig("lr5_plot_probes.png", dpi=150)
    plt.close(fig)

    # --- рисунок 2: длины цепочек и «плохой» размер таблицы -----------------
    ch = res["chains"]
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
    ax = axes[0]
    lens = sorted(int(L) for L in ch["prime"]["hist"])
    ax.bar(lens, [ch["prime"]["hist"][str(L)] for L in lens], color="tab:blue", label="опыт")
    ax.plot(lens, [ch["poisson"][str(L)] for L in lens], "ro-", label="Пуассон (теория)")
    ax.set_xlabel("Длина цепочки")
    ax.set_ylabel("Число ячеек")
    ax.set_title(f"m = {M:,} (простое): длины цепочек".replace(",", " "))
    ax.legend()
    ax = axes[1]
    lens = sorted(int(L) for L in ch["bad"]["hist"])
    ax.bar(lens, [ch["bad"]["hist"][str(L)] for L in lens], color="tab:orange")
    ax.set_yscale("log")
    ax.set_xlabel("Длина цепочки")
    ax.set_ylabel("Число ячеек (лог. шкала)")
    ax.set_title(f"m = {M_BAD:,} (кратно 100): длины цепочек".replace(",", " "))
    fig.tight_layout()
    fig.savefig("lr5_plot_chains.png", dpi=150)
    plt.close(fig)

    # --- рисунок 3 (В6): распределение длин кластеров -----------------------
    cl = res["clusters"]
    fig, axes = plt.subplots(1, 3, figsize=(13, 4.2))
    for ax, a in zip(axes, ["0.6", "0.7", "0.9"]):
        for mode, color, label in (("linear", "tab:red", "линейное"), ("double", "tab:blue", "двойное")):
            hist = cl[a][mode]["hist"]
            xs = sorted(int(L) for L in hist)
            ax.plot(xs, [hist[str(L)] if str(L) in hist else hist[L] for L in xs],
                    "o", ms=3, color=color, label=label)
        ax.set_xscale("log")
        ax.set_yscale("log")
        ax.set_xlabel("Длина кластера")
        ax.set_ylabel("Число кластеров")
        ax.set_title(f"α = {a}")
        ax.grid(True, which="both", alpha=0.3)
        ax.legend()
    fig.tight_layout()
    fig.savefig("lr5_plot_clusters.png", dpi=150)
    plt.close(fig)


# ---------------------------------------------------------------------------
# 9. ВЫВОД РЕЗУЛЬТАТОВ
# ---------------------------------------------------------------------------
def system_info():
    """Строка с характеристиками системы (вставляется в отчёт)."""
    cpu = platform.processor() or platform.machine()
    try:                                          # на Linux модель процессора лежит в /proc/cpuinfo
        with open("/proc/cpuinfo", encoding="utf-8") as f:
            for line in f:
                if line.startswith("model name"):
                    cpu = line.split(":", 1)[1].strip()
                    break
    except OSError:
        pass
    return (f"ОС: {platform.system()} {platform.release()}; процессор: {cpu}; "
            f"Python {platform.python_version()}")


def us(t):
    """Секунды -> микросекунды (для печати)."""
    return t * 1e6
```

```
def print_report(res):
    """Печатает результаты всех экспериментов в консоль."""
    main = res["main"]
    names = {"chain": "Метод цепочек", "double": "Двойное хеширование",
             "linear": "Линейное пробирование", "dict": "dict (эталон)"}
    print(f"\nТаблица 1. n = {N}, m = {M}, alpha = {main['alpha']:.4f}")
    print(f"  {'Реализация':<24}{'вставка, мс':>12}{'успеш., мкс':>13}{'неусп., мкс':>13}"
          f"{'проб усп.':>11}{'проб неусп.':>13}")
    for key in ("chain", "double", "linear", "dict"):
        r = main[key]
        pf = f"{r['probes_found']:.3f}" if "probes_found" in r else "-"
        pm = f"{r['probes_missing']:.3f}" if "probes_missing" in r else "-"
        print(f"  {names[key]:<24}{r['t_insert'] * 1e3:>12.2f}{us(r['t_found']):>13.3f}"
              f"{us(r['t_missing']):>13.3f}{pf:>11}{pm:>13}")
    print("  Теория (проб: успешный / неуспешный):")
    for key in ("chain", "double", "linear"):
        a, b = main["theory"][key]
        print(f"    {names[key]:<24}{a:>8.3f}{b:>10.3f}")

    ch = res["chains"]
    print("\nТаблица 2. Метод цепочек: распределение длин цепочек")
    for key, title in (("prime", f"m = {M} (простое)"), ("bad", f"m = {M_BAD} (кратно 100)")):
        r = ch[key]
        print(f"  {title}: макс. длина {r['max']}, пустых ячеек {r['empty']}, "
              f"занятых ячеек {r['used']}, проб (успешный поиск) {r['probes_found']:.2f}")
    print("  Длина  опыт  Пуассон(теория)")
    for L in sorted(int(x) for x in ch["prime"]["hist"]):
        print(f"  {L:>5}{ch['prime']['hist'][str(L)]:>6}{ch['poisson'][str(L)]:>12.1f}")

    print("\nТаблица 3. Среднее число проб (опыт / теория): неуспешный и успешный поиск")
    print(f"  {'alpha':>6}{'n':>8}  {'линейное неусп.':>22}{'линейное усп.':>20}"
          f"{'двойное неусп.':>20}{'двойное усп.':>18}")
    for a, r in res["sweep"].items():
        print(f"  {a:>6}{r['n']:>8}  "
              f"{r['linear'][1]:>10.2f} /{r['theory_linear'][1]:>8.2f}"
              f"{r['linear'][0]:>10.2f} /{r['theory_linear'][0]:>6.2f}"
              f"{r['double'][1]:>10.2f} /{r['theory_double'][1]:>6.2f}"
              f"{r['double'][0]:>9.2f} /{r['theory_double'][0]:>6.2f}")

    print("\nТаблица 4 (B6). Кластеры занятых ячеек")
    heads = " ".join(f"{bin_label(lo, hi):>6}" for lo, hi in CLUSTER_BINS)
    for a, r in res["clusters"].items():
        print(f"  alpha = {a}, n = {r['n']}, средняя длина при независимом заполнении "
              f"1/(1-a) = {r['random_mean']:.2f}")
        print(f"    {'':<10}{'кластеров':>10}{'средняя':>9}{'макс.':>7}   число кластеров по длинам: {heads}")
        for mode, title in (("linear", "линейное"), ("double", "двойное")):
            s = r[mode]
            print(f"    {title:<10}{s['clusters']:>10}{s['mean']:>9.2f}{s['max']:>7}"
                  f"   {'':<27}" + " ".join(f"{b:>6}" for b in s["bins"]))

    d = res["delete"]
    print(f"\nТаблица 5. Удаление {d['deleted']} ключей из {N}; потеряно из {d['remaining']} оставшихся:")
    print(f"  линейное:  пометка DELETED -> {d['linear_tombstone']},  обнуление -> {d['linear_zero']}")
    print(f"  двойное:   пометка DELETED -> {d['double_tombstone']},  обнуление -> {d['double_zero']}")


def main():
    print("Лабораторная работа № 5, вариант 13")
    print(system_info())
    print(f"Ключи: целые, кратные 100; хеш: k mod m; n = {N}, m = {M}; "
          f"{N_QUERIES} успешных и {N_QUERIES} неуспешных запросов")
    self_test()
    res = {}
    print("\nЭксперимент 1/5: основной ...", flush=True)
    res["main"] = experiment_main()
    print("Эксперимент 2/5: цепочки ...", flush=True)
    res["chains"] = experiment_chains()
    print("Эксперимент 3/5: зависимость от alpha ...", flush=True)
    res["sweep"] = experiment_sweep()
    print("Эксперимент 4/5: кластеризация (B6) ...", flush=True)
    res["clusters"] = experiment_clusters()
    print("Эксперимент 5/5: удаление ...", flush=True)
    res["delete"] = experiment_delete()
    print_report(res)
    make_plots(res)
    print("\nГрафики сохранены: lr5_plot_probes.png, lr5_plot_chains.png, lr5_plot_clusters.png")
    if "--json" in sys.argv:                          # служебный вывод для отчёта
        path = sys.argv[sys.argv.index("--json") + 1]
        with open(path, "w", encoding="utf-8") as f:
            json.dump(res, f, ensure_ascii=False, indent=1)
    if "--show" in sys.argv:
        plt.show()


if __name__ == "__main__":
```

```
main()
```

5. Характеристики вычислительной системы

Параметр	Значение
Процессор	Intel(R) Xeon(R) Processor @ 2.10GHz, 2 логических ядра
Оперативная память	8 ГБ
Операционная система	Linux (Ubuntu 24.04)
Интерпретатор	Python 3.13.18
Таймер	time.perf_counter()

Абсолютные значения времени зависят от системы; число проб, форма распределений и соотношения между реализациями должны сохраниться. Программа при запуске выводит характеристики системы автоматически.

6. Результаты измерений

6.1. Основной эксперимент (n = 30 000, m = 49 999, α = 0,600)

Таблица 2. Время и число проб (10 000 успешных и 10 000 неуспешных запросов)

Реализация	Вставка 30 000 ключей, мс	Успешн. запрос, мкс	Неуспешн. запрос, мкс	Проб усп., опыт	Проб усп., теория	Проб неусп., опыт	Проб неусп., теория
Метод цепочек	28,61	0,40	0,33	1,298	1,300	0,592	0,600
Двойное хеширование	13,81	0,47	0,50	1,568	1,527	2,594	2,500
Линейное пробирование	13,36	0,45	0,63	1,723	1,750	3,621	3,625
Встроенный `dict` (эталон)	1,93	0,06	0,04	—	—	—	—

6.2. Распределение длин цепочек и выбор размера таблицы

Таблица 3. Метод цепочек: длины цепочек при простом m = 49 999 в сравнении с законом Пуассона

Длина цепочки	0	1	2	3	4	5	6
Ячеек (опыт)	27400	16560	4882	976	160	18	3
Ячеек (Пуассон, теория)	27439,7	16464,2	4939,3	987,9	148,2	17,8	1,8

Таблица 4. Влияние размера таблицы на метод цепочек (те же 30 000 ключей, кратных 100)

Размер m	Занятых ячеек	Пустых ячеек	Макс. длина цепочки	Проб (успешный поиск)
49 999 (простое)	22 599	27 400	6	1,10
50 000 (кратно 100)	500	49 500	84	10,94

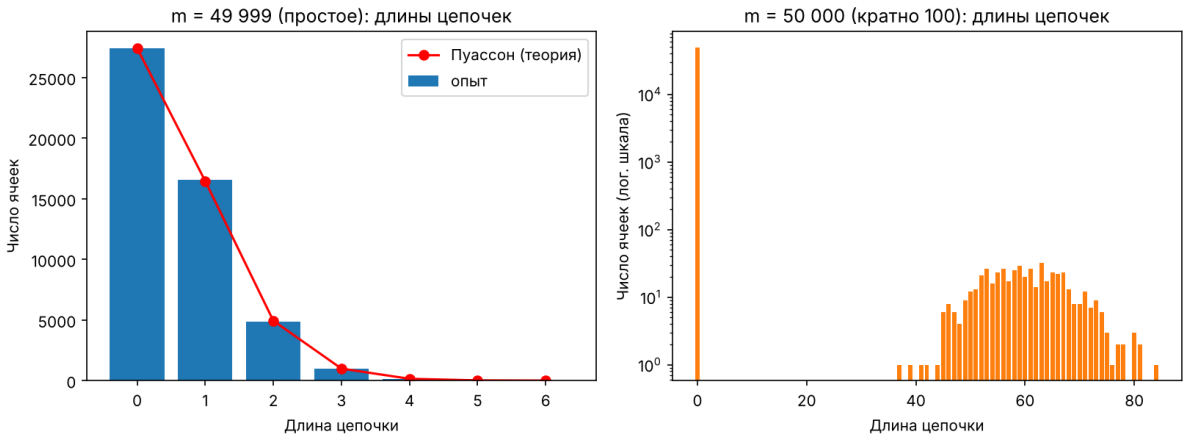


Рис. 6. Распределение длин цепочек: простое m = 49 999 (слева, с наложением закона Пуассона) и m = 50 000 (справа, логарифмическая шкала)

6.3. Зависимость числа проб от коэффициента заполнения

Таблица 5. Среднее число проб, опыт / теория (усреднение по трём наборам ключей, m = 49 999)

α	n	Линейное, неуспешн.	Линейное, успешн.	Двойное, неуспешн.	Двойное, успешн.
0.1	5 000	1,11 / 1,12	1,06 / 1,06	1,11 / 1,11	1,05 / 1,05
0.3	15 000	1,51 / 1,52	1,22 / 1,21	1,43 / 1,43	1,19 / 1,19
0.5	25 000	2,48 / 2,50	1,50 / 1,50	2,04 / 2,00	1,39 / 1,39
0.7	34 999	5,96 / 6,06	2,18 / 2,17	3,48 / 3,33	1,75 / 1,72
0.9	44 999	53,11 / 50,50	5,97 / 5,50	10,77 / 10,00	2,67 / 2,56
0.95	47 499	220,20 / 200,50	10,31 / 10,50	20,97 / 20,00	3,33 / 3,15

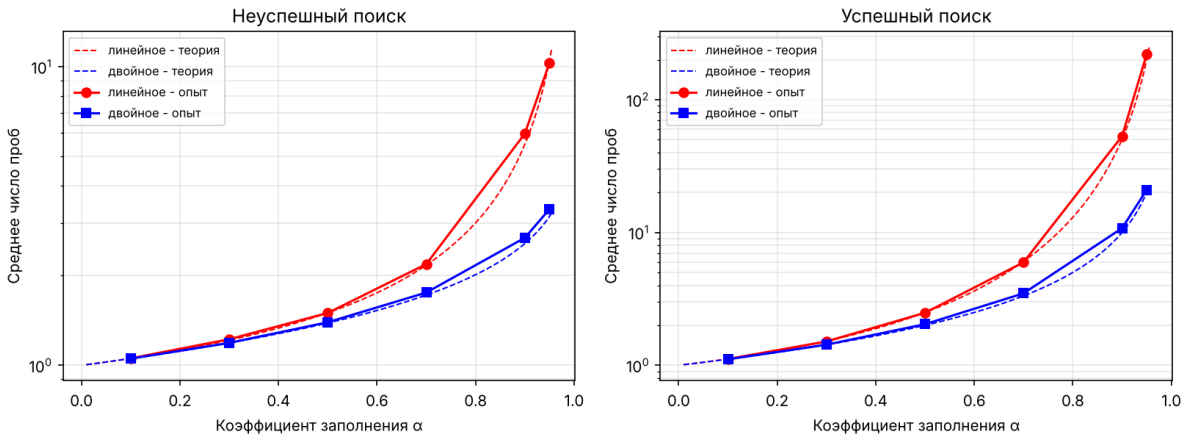


Рис. 7. Среднее число проб в зависимости от α: точки — опыт, штриховые линии — теория (логарифмическая шкала по оси ординат)

7. Дополнительное задание B6: кластеризация при линейном пробировании

Для каждого значения α строилась таблица с n = α·m ключами (для α = 0,6 — ровно n = 30 000), после чего измерялись длины *кластеров* — непрерывных участков занятых ячеек массива. Для сравнения построено то же для двойного хеширования на тех же ключах. Если бы ячейки занимались независимо с вероятностью α, средняя длина кластера составила бы 1/(1 − α) — это значение приведено как ориентир.

Таблица 6. Статистика кластеров

α	n	Способ	Число кластеров	Средняя длина	Ориентир 1/(1−α)	Макс. длина
0.3	15 000	линейное	9014	1,66	1,43	12
		двойное	10425	1,44		8
0.5	25 000	линейное	9839	2,54	2,00	34
		двойное	12508	2,00		14
0.6	30 000	линейное	8972	3,34	2,50	45
		двойное	11985	2,50		20
0.7	34 999	линейное	7517	4,66	3,33	94
		двойное	10472	3,34		32
0.9	44 999	линейное	2969	15,16	10,00	746
		двойное	4470	10,07		108

Таблица 7. Число кластеров заданной длины

α	Способ	1	2	3–4	5–8	9–16	17–32	33–64	65+
0.3	линейное	5700	1908	1093	290	23	0	0	0
	двойное	7226	2233	885	81	0	0	0	0
0.5	линейное	4639	2110	1725	1007	318	39	1	0
	двойное	6189	3227	2331	695	66	0	0	0
0.6	линейное	3644	1755	1654	1180	597	134	8	0
	двойное	4664	2962	2881	1272	203	3	0	0
0.7	линейное	2514	1359	1359	1231	694	289	65	6
	двойное	3006	2248	2678	1967	539	34	0	0
0.9	линейное	734	429	474	424	333	262	154	159
	двойное	406	392	725	1023	1068	718	134	4

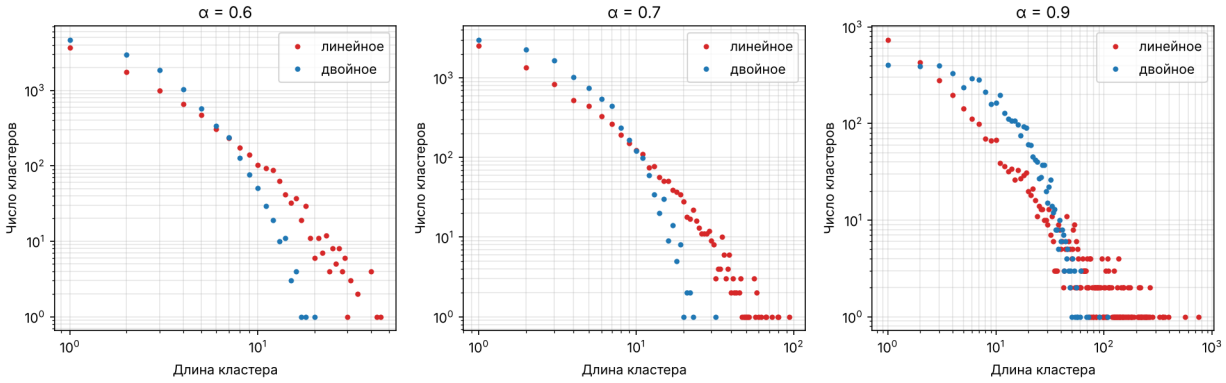


Рис. 8. Распределение длин кластеров (логарифмические шкалы): линейное пробирование и двойное хеширование при α = 0,6; 0,7; 0,9

Контроль удаления.

Из таблицы с n = 30 000 ключами удалено 6 000 ключей (20 %), затем проверялся поиск всех 24 000 оставшихся. При пометке «удалено» потерь нет: 0 (линейное) и 0 (двойное). При простом обнулении ячейки поиск не находит 2579 (10,7 %) оставшихся ключей при линейном пробировании и 2177 (9,1 %) при двойном хешировании — последовательность проб обрывается на «освобождённой» ячейке.

8. Выводы

1. **Измеренное число проб соответствует теории.** При α = 0,60 метод цепочек даёт 1,30 проб при успешном и 0,59 при неуспешном поиске (теория 1,30 и 0,60); двойное хеширование — 1,57 и 2,59 (теория 1,53 и 2,50); линейное — 1,72 и 3,62 (теория 1,75 и 3,63).
2. **Метод деления с простым m хорошо работает на ключах, кратных 100.** Длины цепочек подчиняются закону Пуассона (табл. 3: максимум 6, пустых ячеек 27 400 из 49 999, то есть 55 %, теория 55 %). При размере m = 50 000, имеющем общий делитель со ступенью ключей, используются лишь 500 ячеек, максимальная цепочка достигает 84, а число проб при успешном поиске возрастает с 1,10 до 10,94. Поэтому размер таблицы следует брать простым.
3. **Метод цепочек устойчивее к росту α, открытая адресация чувствительна к заполнению.** Число проб при неуспешном поиске растёт как 1/(1 − α) (двойное хеширование) и ≈ ½/(1 − α)² (линейное): при α = 0,9 опыт даёт 10,8 и 53,1 проб, при α = 0,95 — 21,0 и 220. Двойное хеширование при любых α не хуже линейного пробирования; для метода цепочек число сравнений растёт линейно по α. Именно поэтому таблицы открытой адресации расширяют при α ≈ 0,5–0,75.
4. **B6: линейное пробирование порождает кластеры.** Средняя длина кластера при линейном пробировании выше, чем при независимом заполнении ячеек (1/(1 − α)), и выше, чем при двойном хешировании: при α = 0,6 — 3,34 против 2,50 (ориентир 2,50), при α = 0,9 — 15,2 против 10,1 (ориентир 10,0). Кластеров меньше (8 972 против 11 985 при α = 0,6), а самые длинные намного длиннее: максимум 45 против 20 при α = 0,6 и 746 против 108 при α = 0,9. С ростом α длины растут быстрее, чем 1/(1 − α): распределение длин у линейного пробирования имеет «тяжёлый хвост» (рис. 8), а новый ключ, попавший в кластер, увеличивает его — «богатый богатеет».
5. **Кластеризация объясняет рост числа проб.** Ключ, хеш которого попал в длинный кластер, должен просмотреть почти весь кластер до его конца, а вероятность попасть в кластер пропорциональна его длине. Поэтому при малых α (≈

0,1) линейное пробирование и двойное хеширование не различаются, а при α ≥ 0,5 неуспешный поиск при линейном пробировании заметно дороже (табл. 5). Двойное хеширование «рассеивает» ключи с разными шагами и кластеров практически не образует.

6. **Сравнение с dict и ограничения реализации.** Встроенный dict в 7–15 раз быстрее собственных таблиц при вставке (1,93 мс против 13,81–28,61 мс) и в 8–7 раз при поиске: он реализован на C, а интерпретируемый код тратит время на вызовы методов и работу со списками. Собственная реализация имеет учебное значение: она позволяет считать пробы и видеть цепочки и кластеры. Различия между самими таблицами по времени меньше различий по числу проб, так как стоимость пробы в Python велика и сопоставима со стоимостью вызова метода; метод цепочек медленнее при вставке из-за создания 49 999 списков и кортежей, но быстрее при неуспешном поиске.
7. **Удаление.** При открытой адресации нельзя обнулять ячейку: после удаления 20 % ключей поиск не находил 10,7 % (линейное) и 9,1 % (двойное) оставшихся ключей. Пометка «удалено» полностью устраняет проблему, ценой того, что «надгробия» удлиняют пробы, пока таблица не будет перестроена.
8. **Ограничение хеш-таблиц.** Ключи в таблице не упорядочены, поэтому запросы вида «все ключи из диапазона» или вывод по возрастанию требуют полного просмотра либо другой структуры (упорядоченный массив, дерево).

9. Список литературы

1. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013. — Гл. 11.
2. Кнут Д. Э. Искусство программирования. Т. 3. Сортировка и поиск. — 2-е изд. — М.: Вильямс, 2017.
3. Седжвик Р., Уэйн К. Алгоритмы на Java. — 4-е изд. — М.: Вильямс, 2013. — Гл. 3.4.
4. Документация Python: встроенные типы, dict [Электронный ресурс]. — URL: https://docs.python.org/3/library/stdtypes.html#dict